Assignment 7 – Graphs
Benevento

1. Dijkstra

Node	Shortest Distance	Previous Node
Anya	0	
Frog	∞	
Penguin	∞	
Whale	∞	
Wolf	∞	
Elephant	∞	
Dog	∞	
Rabbit	∞	
Sloth	∞	
Rhino	∞	
Bear	∞	
Bond	∞	

Current: Anya

Visited Nodes: []

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	∞	
Elephant	∞	
Dog	4	Anya
Rabbit	∞	
Sloth	∞	
Rhino	∞	
Bear	∞	
Bond	∞	

Current: Anya

Visited Nodes: [Anya]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	∞	
Elephant	∞	

Assignment 7 – Graphs
Benevento

Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	∞	
Bear	∞	
Bond	∞	

Current: Frog

Visited Nodes: [Anya, Frog]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	4	Whale
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	∞	
Bond	∞	

Current: Whale

Visited Nodes: [Anya, Frog, Whale]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	14	Rhino
Bond	∞	

Current: Rhino

Visited Nodes: [Anya, Frog, Whale, Rhino]

Assignment 7 – Graphs
Benevento

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	14	Rhino
Bond	∞	

Current: Penguin

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	14	Rhino
Bond	7.1	Rabbit

Current: Rabbit

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin, Rabbit]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya

Assignment 7 – Graphs
Benevento

Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	14	Rhino
Bond	7.1	Rabbit

Current: Elephant

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin, Rabbit, Elephant]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	14	Rhino
Bond	7.1	Rabbit

Current: Dog

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin, Rabbit, Elephant, Dog]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	14	Rhino
Bond	7.1	Rabbit

Current: Wolf

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin, Rabbit, Elephant, Dog, Wolf]

Assignment 7 – Graphs
Benevento

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	∞	
Rhino	2	Whale
Bear	8.6	Bond
Bond	7.1	Rabbit

Current: Bond

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin, Rabbit, Elephant, Dog, Wolf, Bond]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Rabbit	2.1	Frog
Sloth	9.1	Bear
Rhino	2	Whale
Bear	8.6	Bond
Bond	7.1	Rabbit

Current: Bear

Visited Nodes: [Anya, Frog, Whale, Rhino, Penguin, Rabbit, Elephant, Dog, Wolf, Bond, Bear]

Node	Shortest Distance	Previous Node
Anya	0	
Frog	0.1	Anya
Penguin	2	Anya
Whale	1	Anya
Wolf	6	Whale
Elephant	2.5	Rhino

Assignment 7 – Graphs
Benevento

3. (text) Comparison [5 points] Which algorithm found the shortest path to H in less iterations Dijkstra or A*? (Explain your answer)

A* found the shortest path to H in fewer iterations than Dijkstra's. Dijkstra's computes the path in all directions using the accumulated cost of the path so far (g), even going along paths that do not go towards the goal, which increased the number of iterations. Since A* uses a heuristic estimate (h) and the cost so far (g), it's able to narrow down the paths and prioritize the shorter paths to H in fewer iterations.

Assignment 7 – Graphs
Benevento

7. Algorithm Analysis

4. WeaklyConnected

Time complexity is O(n) where n is the number of nodes in the graph because the DFS starts at node 0 and in worst case will visit every node once. Even though there's a nested loop checking all possible neighbors, each node is only marked visited one time.

Space complexity is O(n) where n is the number of nodes because we store a visited array of size n. At most, we can visit all nodes.

5. FourCycles

Time complexity is $O(n^4)$ where n is the number of vertices in the graph because the program uses four nested loops (i, j, k, l) to try every possible combination of 4 distinct nodes. Although the code skips cases where nodes are the same, in the worst case it still checks every possible combination.

Space complexity is $O(n^2)$ where n is the number of vertices because we store the graph using an adjacency matrix, which is n × n. All other variables used in the program take constant space.

6. DrawGraph

Time complexity is O(n) where n is the number of nodes in the graph because the program loops through all nodes three times: once to place them in a circle, once to draw edges for each node, and once to draw labels (O(3n) > O(n)). It uses constant math operations to draw the lines.

Space complexity is O(n) where n is the number of nodes because the program stores arrays for the x-coordinates, y-coordinates, and the node labels (O(3n) > O(n)). It also creates an image buffer, but that size is fixed and does not grow with input size. The only part that grows as the input grows is the data stored for each node.

DrawGraphString

Time complexity is $O(n^2)$ where n is the number of nodes because the program builds the adjacency matrix using one loop that runs n times and then prints the matrix using nested loops (n rows and n columns). Since the printing step must visit every cell in the matrix, it takes $O(n^2)$ overall.

Space complexity is $O(n^2)$ where n is the number of nodes because we store the graph using an adjacency matrix, which takes up an $n \times n$ space.